IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 8 June 2025, accepted 30 June 2025, date of publication 4 July 2025, date of current version 28 July 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3585956

RESEARCH ARTICLE

ASIC Design for Real-Time CAN-Bus Intrusion Detection and Prevention System Using Random Forest

JUNSEOK LEE[1], SANGMIN PARK[1], SUA SHIN[1], HYUNGCHUL IM[1], (Graduate Student Member, IEEE), JOOSOCK LEE[2], AND SEONGSOO LEE[1], (Member, IEEE)

[1]Department of Intelligent Semiconductors, Soongsil University, Seoul 06978, South Korea
[2]School of Electronic Engineering, Soongsil University, Seoul 06978, South Korea

Corresponding author: Seongsoo Lee (sslee@ssu.ac.kr)

This work was supported in part by the Research and Development Program of the Ministry of Trade, Industry, and Energy (MOTIE) and Korea Evaluation Institute of Industrial Technology (KEIT) under Grant RS-2022-00154973, Grant RS-2023-00232192, Grant RS-2024-00403397, and Grant RS-2022-00155731; and in part by the Technology Innovation Program of MOTIE and KEIT under Grant RS-2024-00433615.

ABSTRACT A controller area network (CAN) is the most widely used protocol to communicate between electronic control units (ECUs) inside a vehicle. However, it lacks basic security functions and a structure vulnerable to external attacks. Therefore, it is important to develop an intrusion detection system (IDS) that can effectively identify malicious and legitimate CAN messages, as well as an intrusion prevention system (IPS) that can block such malicious messages in real time. In this study, a random-forest-based intrusion detection system (RF-IDS) was proposed and integrated with a CAN controller to design a system that can perform real-time detection and attack blocking. Existing IDS studies aim for high detection performance. However, the real-time blocking function is limited or the possibility of hardware implementation is yet to be sufficiently verified. To address this issue, we designed an IDS that can immediately determine if an attack is present when receiving a CAN frame and blocking the attack node. Fast and efficient detection is possible complex matrix operations because the random forest model performs detection based on comparison operations. This confirmed that the corresponding structure achieved an average detection time of 1.86 μs and detection accuracy of more than 99%, allowing real-time intrusion blocking. Finally, hardware implementation was achieved by completing the design of the ASIC chip through a multiproject wafer process.

INDEX TERMS Application-specific integrated circuit, controller area network, intrusion detection system, intrusion prevention system, machine learning, random forest.

I. INTRODUCTION

The recent rapid developments in vehicle technology have led to a rapid increase in the number of ECUs in vehicles [1]. Advancements in vehicle technologies such as driving assistance systems and autonomous driving functions had led to the use of cameras, lidars, and radars to vehicles. Accordingly, vehicles have transformed beyond simple means of transportation into complex and sophisticated electronic systems. The threat to vehicle security continues to increase because ECUs in vehicles can exchange data through V2X communication systems connected to external networks [2]. This expansion can increase the convenience and functionality of the vehicle while increasing the likelihood of exposure to external attacks such as hacking. Miller and Valasek [3] demonstrated that this vulnerability can be exploited to hack a radio system, intrude into an in-vehicle network, and transmit malicious commands to a CAN bus. This indicated that the main control system of a vehicle can be affected by the simple hacking of the radio system,

The associate editor coordinating the review of this manuscript and approving it for publication was Angelo Trotta.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

and core functions such as acceleration, braking, and steering can be manipulated remotely. Thus, research into solving security vulnerabilities in vehicle networks has become increasingly important.

Various protocols such as CAN, FlexRay, local interconnect network (LIN), media oriented systems transport (MOST), and automotive ethernet are used in combination for vehicle network communication. However, the most commonly used protocol is CAN, which is a communication standard for sending and receiving data between ECUs in real time. CAN is responsible for the overall movement of a vehicle, from key systems, such as engine control, brake systems, and airbags, to minor parts. However, the CAN protocol is vulnerable to security issues because the message is broadcast to all nodes and does not contain information about the sending node [4]. Reference [5] showed that the vehicle was exposed to several external attack vectors, such as OBD-II ports, media players, Bluetooth, satellite radio, and telematics units. Telematics are directly connected to CAN buses through cellular networks and can control functions such as GPS and hands-free audio, which suggests that exploiting them by external attackers can pose a serious security threat. Various IDS have been studied to improve vehicle-network security to address this security vulnerability. Existing IDS studies have focused on achieving high detection performance. however, there is a problem in that the consumption of hardware resources is large or the real-time response is difficult.

In this study, a RF-IDS capable of real-time blocking was implemented as a ASIC to overcome the limitations of existing methods. For real-time operation of in-vehicle edge devices, a lightweight random forest model was selected and integrated into the CAN controller, which enables the real-time acquisition of CAN data can be required for IDS decision-making during CAN frame reception. Experimental results indicated that the RF-IDS model designed in this manner achieved an average detection latency of 1.86 µs, which demonstrates that real-time blocking is feasible with sufficient margin even at the maximum transmission speed of CAN communication. Finally, MPW was taped out and proceeded with TSMC 180 nm processes.

To the best of our knowledge, prior research on IDS for in-vehicle network communication implemented as a physical chip for real-world deployment is lacking. Therefore, unlike existing IDS studies, this paper proposes a model that simultaneously considers hardware optimization and real-time detection performance, which demonstrates the feasibility of a practical security solution applicable to in-vehicle edge devices.

The major contributions of this study are as follows:

1) We propose a random forest-based IDS model that performs predictions through simple comparison operations, which enable efficient operation even in resource-constrained environments. Furthermore, Area consumption is minimized by adopting a lightweight design that utilizes simple bitwise operations instead of floating-point operations.
2) A real-time intrusion prevention function is implemented by optimizing the detection speed of the IDS. The system is designed to minimize detection latency and enable an immediate response to intrusions by utilizing fast comparison operations and parallel processing capabilities of the random forest model.
3) We propose a hardware design with a compact area and low computational overhead through ASIC implementation using MPW. In this study, tape-out was conducted using the TSMC 180 nm process, and the IDS, including the synthesized CAN module, was measured to have a total gate count of 245,222. In addition, a hardware architecture was designed to optimize memory resource utilization by integrating five 4 KB static random-access memories (SRAMs).

The remainder of this paper is organized as follows: Section II reviews related works, and Section III outlines the background of the study. Section IV describes the proposed method, including the system architecture and detection mechanism. Section V presents experimental results for both FPGA and ASIC implementations, along with a comprehensive analysis. Section VI discusses limitations of the study, and Section VII concludes the study.

II. RELATED WORKS

Extensive research is being conducted on various intrusion-detection systems to enhance CAN security in response to the increase in security threats to in-vehicle networks. Although the CAN enables real-time communication between electronic control units within a vehicle, it remains vulnerable to cyberattacks because of the absence of message authentication and encryption mechanisms. Consequently, various CAN IDS that incorporate diverse security strategies have been explored actively. The significance and ongoing relevance of this research topic are evident in studies cited in [6], [7], [8], and [9]. Recently, real-time detection systems utilizing hardware accelerators have been developed in addition to the software-based IDS utilizing machine and deep learning.

Existing research has focused on enhancing the IDS detection performance by advancing various algorithms. Seo et al. proposed a method to address the vulnerability of machine learning models by utilizing a generative adversarial network (GAN), preventing the evasion of an ML-based IDS. Bhavsar et al. introduced a federated learning-based IDS (FL-IDS) that integrated logistic regression and convolutional neural networks (CNNs) [11]. Instead of centralizing data collection, their approach trained models directly on individual edge devices while sharing only model updates, improving privacy and reducing data transmission costs. However, the distributed nature of training increased the computational overhead and slowed the detection speed. Yang et al. proposed MTH-IDS, which employed hybrid machine-learning techniques to enhance detection performance. However, the system lacked sufficient optimization

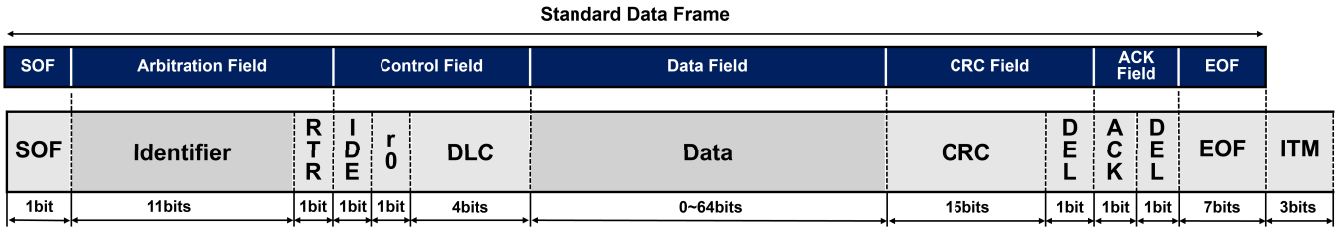


FIGURE 1. Structure of the CAN standard frame format.

for real-time detection speeds because of its multistage learning process [12].

Existing studies primarily focused on enhancing the detection performance through algorithmic improvements, relatively little attention has been paid to real-time performance and the feasibility of hardware implementation. Song et al. highlighted these issues, emphasizing that real-time response speed and hardware implementation potential are critical factors in addition to detection accuracy [13]. Recent research has explored the implementation of IDSs using FPGA-based hardware accelerators to address the limitations of software-based IDS. FPGA-based IDS aims to accelerate neural network models such as CNN, multilayer perceptron (MLP), and binarized neural networks (BNN) to enhance detection performance. Rangsikunpum et al. utilized a BNN to significantly improve the computational efficiency of an FPGA [14]. Their study achieved a detection speed of 0.259 ms per frame using Xilinx Zynq-7000, focusing on reducing the computational complexity with a lightweight model compared to that of a CNN. However, immediate detection remains challenging in in-vehicle network environments, wherein real-time responses are crucial. Kandelwal and Shreejith proposed an approach for accelerating quantized convolutional neural networks (QCNN) on an FPGA to improve the IDS performance [15]. However, CNN-based methods have practical limitations in vehicle ECU environments because of their high computational requirements and excessive lookup table (LUT) consumption by FPGA, which leads to a significant hardware resource overhead.

III. THEORETICAL BACKGROUND

A. CONTROLLER AREA NETWORK

1) CAN PROTOCOL OVERVIEW

The CAN protocol is a serial communication standard designed for efficient and reliable data exchange between various ECUs in a vehicle, and it was established as the ISO 11898 standard in the 1980s. This protocol has been widely adopted in the automotive industry because of its suitability for fast and stable communication [16], [17]. The CAN protocol has evolved into multiple versions, including CAN 2.0A, CAN 2.0B, and CAN FD, to satisfy the diverse requirements of in-vehicle networks. Each version introduces variations in the frame format, data length, and transmission speed, which enhances the overall flexibility of the network.

CAN 2.0A is the initial CAN standard that utilizes the standard frame format and determines message priority based on an 11-bit ID. This design enabled efficient data transmission and reception between ECUs in relatively simple network environments. The data frame supports a maximum payload of 8 bytes with a maximum transmission speed of 1 Mbps.

CAN 2.0B extends CAN 2.0A by adopting an extended frame format that enables using a 29-bit ID, thereby achieving a larger range of message IDs. This scalability was developed to accommodate complex data exchange between ECUs in large-scale networks.

CAN FD, which is the latest CAN standard, was introduced by Bosch in 2012 for addressing the limitations of existing CAN protocols and meeting the demands of modern vehicle networks. Key enhancements in the CAN FD include support for data payloads of up to 64 bytes and increased transmission speeds of up to 8 Mbps through bitrate switching in the data phase. In addition, the cyclic redundancy check (CRC) field length was extended to enhance data integrity.

The design used in this study was based on CAN 2.0A. The structure of the standard frame format used in CAN 2.0A is illustrated in Fig. 1. The start of frame is a single-bit field that marks the beginning of a frame and synchronizes data transmission. The ID field defines the message ID and priority, where a lower numerical value indicates a higher priority, thereby contributing to bus arbitration. The control field contains configuration information for the data frame, including essential metadata such as the data length code. The data field carries actual transmitted data and supports up to eight bytes (64 bits). The CRC field contains an error-detection code used to verify data integrity. The acknowledgement (ACK) field ensures successful message reception with the receiver sending a response signal if no errors are detected. The end of frame (EOF) marks the conclusion of the frame, while the intermission (ITM) provides the necessary idle period before the next frame transmission begins.

2) CAN ARBITRATION MECHANISM

In CAN communication, the arbitration mechanism prevents data collisions by determining message priority when

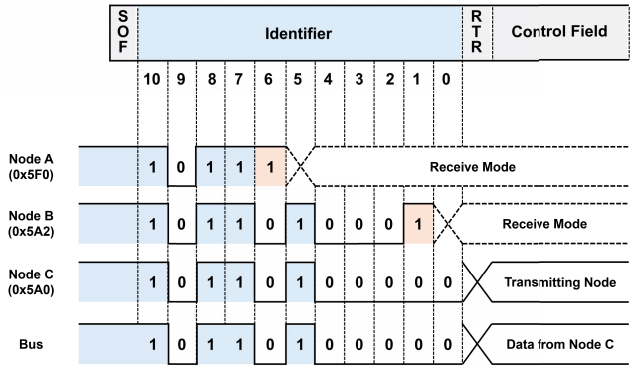


FIGURE 2. CAN arbitration process.

multiple nodes connected to the CAN bus attempt to transmit data simultaneously in a multimaster configuration. This arbitration process follows a bitwise arbitration method in which message conflicts are resolved using dominant and recessive bits. In the CAN protocol, a transmitted zero is defined as the dominant bit, while a one is considered a recessive bit. Owing to the physical characteristics of the CAN bus, the dominant bit takes precedence and is registered on the bus if dominant and recessive bits are transmitted simultaneously at the same location. Consequently, each node continuously monitors the bits on the CAN bus in real time when multiple nodes begin transmissions simultaneously. If the bits transmitted in the ID field match the bits observed on the bus, they retain priority and continue the transmission. However, if a node detects a dominant bit on the bus while transmitting a recessive bit in the ID field, it immediately ceases transmission and switches to the reception mode, waiting for the higher-priority message to be transmitted first. Through this arbitration process, high-priority messages always take precedence, thereby ensuring that all nodes can successfully complete data transmission without collisions.

Fig. 2 illustrates an example of an arbitration procedure. When nodes A, B, and C attempt to transmit simultaneously, Node A, which transmits a recessive bit at ID bit 6 detects a dominant bit on the bus and immediately halts the transmission, switching to the reception mode. Nodes B and C continue their transmissions. Subsequently, Node B transmits a recessive bit at ID bit 1. however, it detects a dominant bit on the bus, which lead it to stop transmission and switch to the reception mode. Consequently, Node C, which has the highest priority, retains control of the bus and successfully completes its transmission.

3) CAN ERROR MANAGEMENT AND STATE TRANSITIONS

The node that detects the error transmits an error flag when an error occurs in CAN. The transmit error counter (TEC) increases by eight if the node is in the transmission state, while the receiver error counter (REC) increases by one if the node is in the reception state. The error states were classified into three levels: error active, error passive, and bus-off,

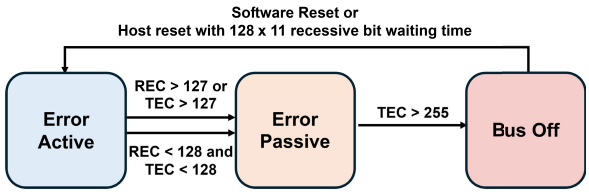


FIGURE 3. State transition as the error counter changes.

with each state determined based on the values of TEC and REC. As illustrated in Fig. 3, the behaviors of the three error states are categorized and described as follows.

- **Error active state:** In the error active state, the node operates normally, and upon detecting an error, it transmits an error flag consisting of six dominant bits.
- **Error passive state:** If the TEC or REC exceeds 128, the node transitions to the error passive state. In this state, the node transmits a passive error flag when an error is detected. In addition, upon resuming data transmission, the node must observe an additional suspension period of 8 bits beyond the standard 3-bit ITM shown in Fig. 1, which makes it difficult to maintain priority in the arbitration process.
- **Bus-off state:** If the TEC exceeds 255, the node enters the bus-off state, wherein it is completely disconnected from the network and cannot transmit or receive data. To recover from the bus-off state, the node must detect 11 consecutive recessive bits on the bus at least 128 times, at which point both TEC and REC are reset to 0, thereby allowing the node to rejoin the communication process. In addition, the TEC and REC decrease by 1 per successful transmission if no further errors occur and normal data transmission and reception continue, thereby enabling the system to automatically recover over time.

B. CAN ATTACK MODEL

Attacks on CAN buses occur through unauthorized access to in-vehicle networks or by manipulating communication between ECUs. The CAN protocol is inherently vulnerable to external attacks because its initial design prioritizes real-time data transmission and reception over security. Such attacks can be executed remotely via wireless connections in addition to wired interfaces such as the OBD-II port [18]. In addition, the broadcast communication mechanism of the CAN presents significant security vulnerabilities. All ECUs share the same bus for message transmission and reception, and therefore, an attacker can intercept, manipulate, or inject messages targeting specific ECUs, potentially causing malfunctions in the vehicle system. Several attack techniques exploit these vulnerabilities, including denial-of-service (DoS), spoofing, and fuzzy attacks, each with distinct characteristics. Figs. 4–6 illustrate representative scenarios and mechanisms these attack techniques.

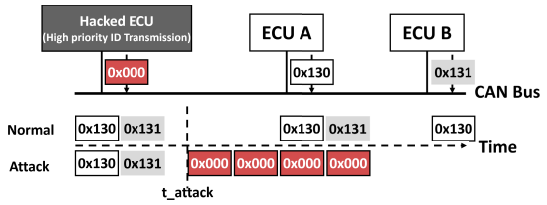


FIGURE 4. DoS attack scenario in the CAN.

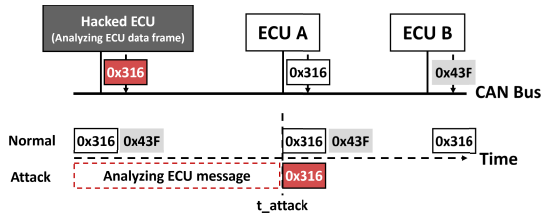


FIGURE 5. Spoofing attack scenario in the CAN bus.

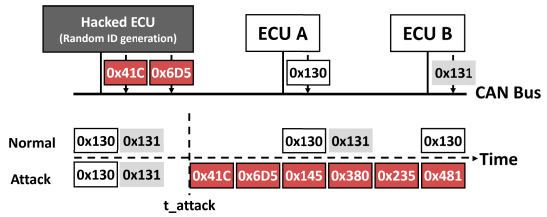


FIGURE 6. Fuzzy attack scenario in the CAN bus.

1) DoS ATTACK

DoS attacks exploit the CAN bus arbitration mechanism to disrupt network functionality by continuously transmitting high-priority messages [19]. In a DoS attack, an attacker repeatedly sends messages with the highest-priority IDs at extremely short intervals, which prevent other ECUs from gaining access to buses. Consequently, critical vehicle systems such as engine control and braking systems fail to function properly, thereby posing a serious threat to vehicle safety. For example, an attacker can periodically inject high-priority messages into a CAN bus. In this scenario, normal ECUs would not be able to access the bus, thereby rendering essential vehicle functions unresponsive and potentially leading to system-wide failures.

2) SPOOFING ATTACK

Spoofing attacks involve generating false messages to deceive specific ECUs and accept them as legitimate data [20]. An attacker can manipulate critical vehicle parameters such as RPM, speed, and gear state, thereby causing the vehicle to operate based on falsified state information. This can lead to erroneous system behavior, which compromises both functionality and safety. The procedure for a spoofing attack is as follows:

1) The attacker observes CAN bus traffic for a period of time to analyze normal message patterns.

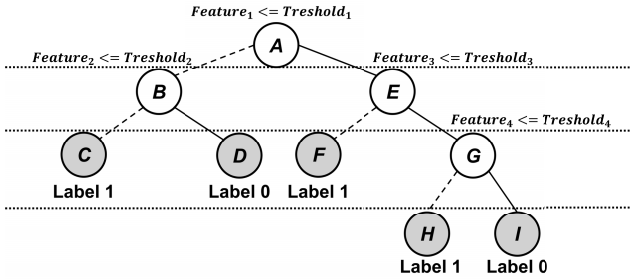


FIGURE 7. Decision tree structure.

2) The attacker injects forged messages onto the CAN bus using the same message ID as a legitimate ECU.
3) The compromised ECU processes the false data, which causes other ECUs to react based on manipulated information.

For example, an attacker can alter vehicle speed data, thereby causing the instrument cluster to display an incorrect speed or modify gearshift messages, potentially forcing an unintended gear change.

3) FUZZY ATTACK

Fuzzy attacks involve injecting random data into CAN messages to trigger unexpected behaviors in the ECUs [21]. In such an attack, the attacker transmits arbitrary or malformed data to the CAN bus, forcing ECUs to process unpredictable inputs, thereby leading to system instability. The potential consequences of a fuzzy attack include the following:

- Failure of ECU functionality caused by errors in processing unexpected input data.
- Misinterpretation of input values, leading to abnormal behavior in vehicle systems.
- Discovery of ECU vulnerabilities through the repeated injection of random data, which can be exploited in further targeted attacks.

For example, certain ECUs may misinterpret the message as a valid command if an attacker injects randomized data values, which can result in the unintended activation of vehicle functions. In severe cases, the ECU may crash or reboot, disrupting critical vehicle operations.

C. RANDOM FOREST

Random forest is an ensemble learning technique that constructs multiple independent decision trees and combines their outputs to enhance prediction performance and stability [22]. Each decision tree partitions data based on specific features and threshold values, and training is performed using different subsets of data samples.

Fig. 7 illustrates the structure of the individual decision trees. Each node in the tree splits the data into two groups by comparing feature values with a predefined threshold. For example, the root node (A) determines whether to route data to the left or right subtree by comparing feature $_1$

with threshold $_1$. This process continues through intermediate nodes until a leaf node (e.g., C, D, F, H, or I) is reached, where the final classification or decision is assigned. In a random forest model, these individual trees make independent predictions, and the final output is determined through a majority voting mechanism. This ensemble approach reduces variance, improving prediction stability compared to that of a single decision tree. In addition, random sampling and random feature selection techniques are applied to minimize correlations between trees, which enhance generalization performance and robustness of the model.

IV. PROPOSED ARCHITECTURE

A. SLIDING WINDOW

In a CAN, every message is assigned a unique ID that determines message priority and facilitates communication between specific ECUs. Under normal operating conditions, the same ID is transmitted at regular intervals to form a consistent communication pattern. however, abnormal patterns such as irregular changes in the frequency or order of specific IDs can emerge during an attack. This study employs a sliding window technique to analyze the continuous variations in CAN ID sequences to detect such abnormalities.

A sliding window is a data preprocessing technique that continuously updates a fixed-size window by incorporating new data while excluding older data [23]. A sliding window is used in this study to track changes in CAN ID patterns over time, thereby enabling the detection of anomalies by identifying deviations from normal communication patterns. Fig. 8 presents performance according to the size of each sliding window. The detection performance improves up to a window size of 11. Since the window size is proportional to the number of bits required to store parameters, it is also proportional to hardware resource consumption. In this study, we selected a window size of 11 by considering the trade-off between detection performance and hardware resource consumption. Fig. 9 shows a visual representation of the sliding window operation. In a normal CAN, the order and frequency of message IDs remain consistent as the sliding window moves, ensuring that no particular ID unexpectedly disappears or occurs with abnormal frequency. However, deviations from the expected pattern can be detected in the presence of an attack.

Typical anomalies that can indicate an attack include the following.

- **DoS attack:** Unusually high frequency of certain CAN IDs.
- **Spoofing attack:** Irregular reordering of CAN IDs.
- **Fuzzy attack:** Appearance of an unknown CAN ID that does not typically exist in the network.

As shown in Fig. 9, In Seq2, when an ID_{12} enters the sliding window, the IDS compares the updated sequence with the previous pattern to identify changes in the order or frequency as potential anomalies. For example, various

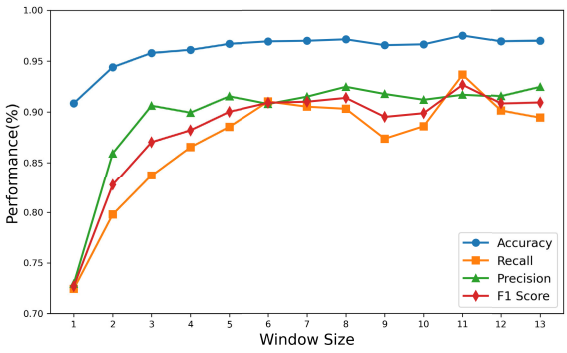


FIGURE 8. Detection performance by window size.

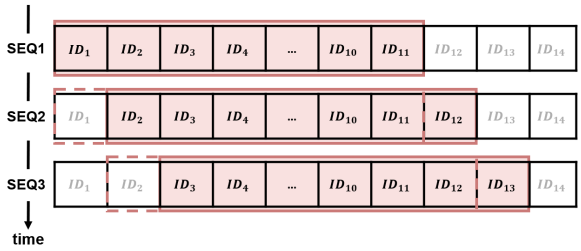


FIGURE 9. Example of the sliding window.

CAN IDs are transmitted at regular intervals under normal communication conditions. However, during a DoS attack, a specific ID—such as 0 × 000—can dominate the CAN traffic by appearing at an abnormally high frequency. An IDS detects these deviations from the expected pattern and identifies them as signs of an attack. Unlike simple inspection of individual IDs, sliding window-based pattern analysis enables the IDS to learn the normal communication flow of the CAN network and detect anomalies more effectively. This enables the random-forest-based IDS to continuously learn CAN ID sequences over time and perform real-time intrusion detection with improved accuracy.

B. BASIC FLOW OF RF-IDS

The proposed RF-IDS detects anomalies by analyzing CAN ID patterns using a trained random forest model and is designed to perform real-time intrusion detection and blocking. Fig. 10 illustrates the overall operational process of the proposed RF-IDS.

In the offline field, a training dataset that includes both normal and attack data is extracted from the original CAN traffic and used to train a random forest model. During this process, features are extracted based on the CAN ID sequences, thereby enabling the random forest model to learn structured patterns. Consequently, a trained decision-tree ensemble is generated.

The online phase begins, incoming CAN messages from the in-vehicle network are processed in real time. The feature extraction module generates a sliding window of CAN ID sequences to capture real-time communication patterns.

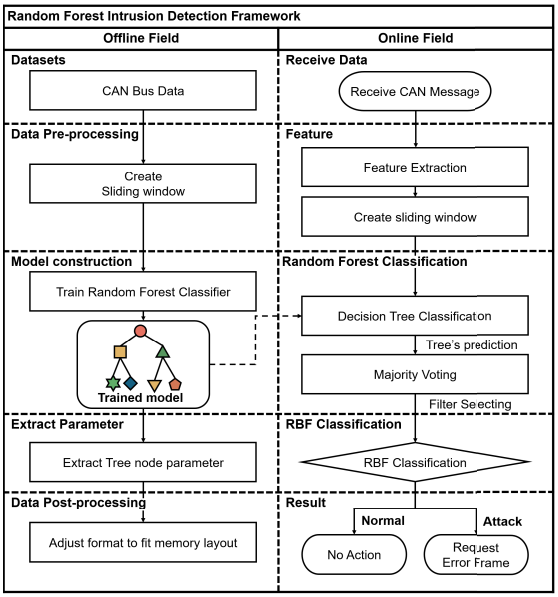


FIGURE 10. Flowchart of the intrusion detection process.

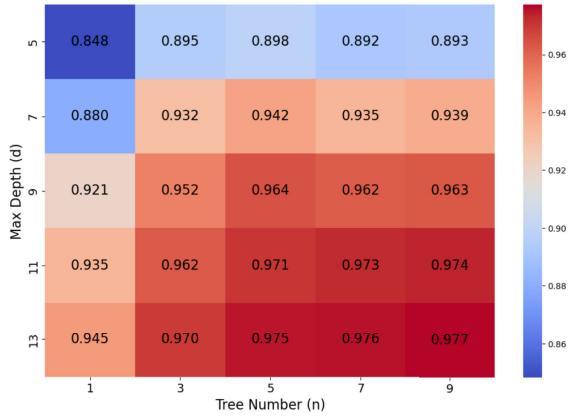


FIGURE 11. Detection accuracy heatmap for random forest parameters.

These sequences are then input into the trained random forest classifier. Each decision tree performs an individual classification, and the results are aggregated through majority voting. At this stage, the random forest functions as a multi-class classifier capable of distinguishing between various attack types. However, these predictions are subsequently verified by a rule-based filter (RBF) [24], which converts the result into a binary classification either normal or attack. Through this cross-check mechanism, the random forest based IDS achieves higher detection accuracy and fewer false positives.

C. RANDOM FOREST PARAMETER OPTIMIZATION

The performance of the random forest model and the hardware resources required for implementation must be considered together. In the random forest model, the detection performance improved with an increase in tree depth. however, the hardware resource consumption also increased exponentially. In a random forest, the number of trees(n) represents the number of individual decision trees trained by the model, and the tree depth (d) indicates the maximum number of layers in each tree. In this case, the maximum number of nodes in the random forest is $n \times (2^d - 1)$, and the number of nodes increases exponentially with an increase in tree depth.

Fig. 11 presents a heat map that visually represents the detection accuracy based on the number of trees (n) and tree depth (d) obtained using GridSearchCV. The horizontal axis represents the number of trees (n), the vertical axis represents the tree depth (d), and darker shades of red indicate higher detection performance.

As a result of the experiment, the detection performance improved with an increase in the tree depth. however, beyond depth 9, the improvement gradually decreased and eventually saturated. Furthermore, the number of nodes increased exponentially with an increase in the tree depth, leading to a higher area consumption and computational burden. In other words, while detection performance continues to improve at a slower rate, computational and memory requirements keep increasing, highlighting the need to balance detection accuracy and hardware cost.

Moreover, the effect of increasing the number of trees (n) on performance improvement decreases more rapidly beyond a certain threshold. Although increasing the number of trees enhances the detection performance, the performance improvement slows down more quickly as tree depth increases beyond a certain threshold. In other words, increasing the number of trees beyond a certain point leads to higher computational cost without a meaningful improvement in detection performance, making the design less efficient.

Based on these results, the optimal values were selected as 9 for the tree depth (d) and 5 for the number of trees (n). This configuration was selected to achieve a balanced structure that minimized the area and computational resource consumption while preserving the detection performance.

D. HARDWARE ARCHITECTURE

The random forest model consists multiple decision trees, where each tree independently classifies the data. The final detection result is determined through a majority voting mechanism. This study proposes an approach for maximizing the real-time detection performance by optimizing the random forest model for hardware implementation. The proposed method enables real-time intrusion detection even in the constrained network environment of a vehicle by designing an efficient architecture that reduces computational complexity while enhancing detection speed.

1) RANDOM FOREST DECISION PROCESS

Each individual decision tree in a random forest compares the value of the feature of the input data from the root node with the value stored therein for classification. Fig. 12 shows the process by which a random forest classifies the input data

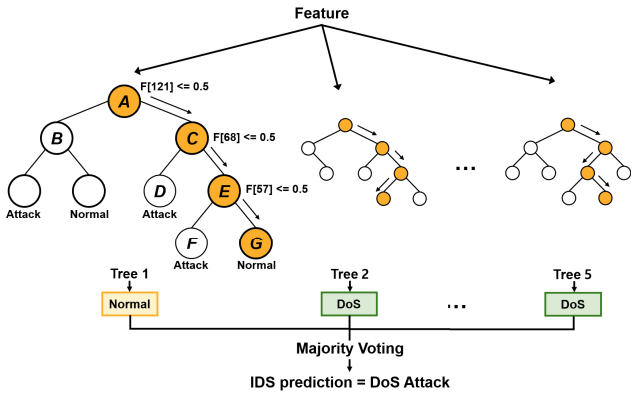


FIGURE 12. Multi-classification process of RF-IDS.

Node	Address of next true node	Address of next false node	Class	Determined	Index of feature
Node A (non-leaf)	B_{Addr}	C_{Addr}	0	0	121bit of Feature
Node G (leaf)	0	0	1	1	0

FIGURE 13. Node parameter for hardware implementation.

as normal or attack data. Meanwhile, in a general decision tree model, the feature value is stored as a real number, and each node performs a comparison operation of feature $\leq$ threshold for different features and thresholds. Thus, each node must store a real value of the same size as the feature and a threshold to perform a comparison.

In this study, the threshold was fixed to 0.5 by training the model to compare and calculate a specific bit among 121 bits of features generated through the sliding window. Therefore, each node can classify data by storing only a 7-bit feature index, indicating the location of a specific bit to be used in the comparison operation rather than by storing a 121-bit threshold. Consequently, classification at each step requires only a single-bit comparison, which simplifies the operation at each node. The leaf node stores the final detection result when the process from the root to the leaf node is complete. This approach offers the following advantages in terms of hardware implementation: reduced memory usage, minimized computational complexity, and optimized real-time processing.

1) Memory usage reduction: Compared to conventional multi-bit feature storage methods, this approach significantly reduces memory space by storing only a feature index that indicates the location of the specific bit used for comparison operations.
2) Simplifying operations and improving detection speed: The delay occurring during the tree search process is reduced by simplifying the comparison operation, thereby enabling more efficient real-time detection. The threshold value is fixed at 0.5, eliminating the need for floating-point operations and allowing faster hardware execution through simple bitwise comparisons.

Fig. 13 illustrates the parameter format used to store the node information for each tree in the random forest model. This node structure is designed to facilitate efficient classification operations, including essential information such as the next node address, class information, and feature indices for comparison operations. The tree search process can be simplified into a memory-reference operation by employing this parameter structure, thereby eliminating the need for complex computations. This approach effectively balances the detection performance and processing speed. The parameters are defined as follows:

- **Address of the next true node:** The address of the next node to transition to when the bit corresponding to the feature index does not satisfy the condition (e.g., below the reference value).
- **Address of next false node:** The address of the next node to transition to when the bit corresponding to the feature index satisfies the condition (e.g., above the reference value).
- **Class:** This parameter represents the classification result of the tree. A value of 1 indicates an attack, while 0 indicates normal traffic. This value is valid only when a final decision has been made at the corresponding tree (i.e., Determined = 1).
- **Determined:** A value of 0 indicates that the current node has a child node and the search must continue to the next node. A value of 1 indicates that the current node is a leaf node, meaning the classification decision has been reached.
- **Index of feature:** This index specifies the position of the bit to be used for the comparison operation within the 121-bit feature vector. For example, in Fig. 13, Node A is a non-leaf node that performs a comparison operation based on the 121st feature bit. According to the comparison result, it transitions to either Node B or Node C. At this stage, Node A is assigned a determined value of 0, which indicates that the final classification decision is yet to be made. Conversely, Node G is a leaf node that determines the detection result as an attack. Since no further traversal is required, all next node addresses are set to 0. The determined value is 1, and the class value is also 1, indicating that the input is classified as an attack.

This is optimized such that tree exploration can be treated as a simple memory reference operation and maintain a balance between the detection performance and computational speed.

2) SYSTEM ARCHITECTURE

Fig. 14 shows the final prediction process in which the multi-class classification result from the random forest is converted into a binary classification through the RBF. In the random forest, only the CAN ID is used as the input feature, and the output is classified into one of four categories: DoS, Fuzzy, Spoofing, and Normal. The intermediate decision class is cross-checked with the corresponding attack-specific filter, and finally determined as either normal or attack. If the intermediate result is classified as normal, it is

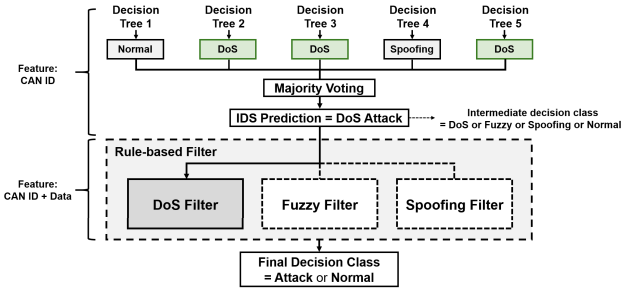


FIGURE 14. Final prediction process of the proposed IDS architecture.

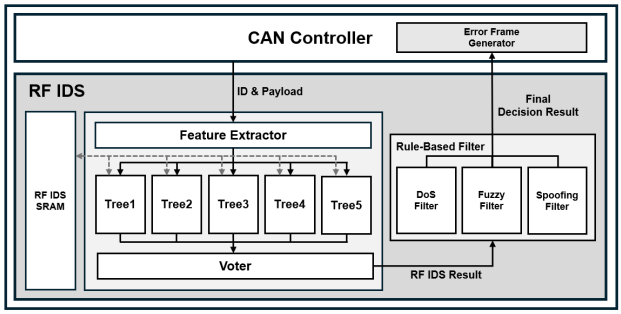


FIGURE 15. Random forest based IDS architecture.

cross-checked against all filters. At this stage, the RBF uses both the CAN ID and Data fields as input features. Fig. 15 illustrates the overall hardware architecture of the proposed RF-IDS. The system comprises a feature extractor, a random forest decision module, an RBF, a module to output the final detection results, and error frame generator.

The detection process is as follows: The CAN ID and payload data collected from the vehicle network are input into the feature extractor, which applies a sliding window technique for generating a 121-bit ID feature for IDS processing. The generated feature is transmitted to the IDS, where detection is performed using the tree data stored in SRAM. The tree module processes data based on the node information retrieved from SRAM and generates a detection result. The voter module then aggregates the results from all trees and forwards them to the RBF for additional verification. The RBF, which comprises DoS, spoofing, and fuzzy filters, refines detection results by validating and adjusting the classification for different types of attacks. Then, the final classification is output as a decision result signal, which categorizes the traffic as normal or attacked.

If an attack is detected, the CAN controller with the integrated IDS immediately generates an error frame, thereby increasing the TEC of the attacking node. If the TEC exceeds a certain threshold, the node transitions to a bus-off state, effectively blocking it from the network, as illustrated in Fig. 3. Unlike conventional IDS solutions that provide only passive detection, this approach enables an IPS function, actively blocking attacks in real time within a CAN network. Further, the TEC gradually decreases if normal data transmission resumes, thereby enabling the affected nodes to recover automatically and rejoin the network.

V. EXPERIMENTS AND ANALYSIS

1) PERFORMANCE METRICS

Representative performance metrics such as accuracy, recall, precision, and F1 score were used to evaluate the performance of the IDS. These metrics quantitatively assess the reliability of the detection model, with each metric providing insight into different aspects of detection performance. These metrics are defined as

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN} \tag{1}$$

$$Precision = \frac{TP}{TP + FP} \tag{2}$$

$$Recall = \frac{TP}{TP + FN} \tag{3}$$

$$F1 - score = \frac{2 \cdot Recall \cdot Precision}{Recall + Precision} \tag{4}$$

- **Accuracy:** This metric represents the overall proportion of correctly classified instances, including both normal and attack data. Although high accuracy suggests that the IDS effectively analyzes traffic patterns, it is not a reliable performance measure in imbalanced datasets where normal messages significantly outnumber attack messages. In such cases, the model can achieve high accuracy simply by predicting the majority class (normal traffic), thereby limiting its effectiveness in assessing IDS performance.
- **Precision:** This metric quantifies the proportion of actual attacks among instances classified as attacks by the IDS. A higher precision indicates fewer FPs, which implies that the IDS minimizes unnecessary alerts and prevents the excessive blocking of normal traffic. However, a low recall may result in undetected attacks even if precision is high, thereby necessitating a balance between precision and recall for a comprehensive evaluation of IDS reliability.
- **Recall:** This metric measures the proportion of actual attacks correctly detected by the IDS. A higher recall indicates fewer false negatives (FNs), making it a critical metric for evaluating the effectiveness of IDS in detecting attacks from a security perspective. However, only a high recall value does not guarantee optimal performance because frequent false positives (FPs) can lead to excessive false alarms in real-world applications.
- **F1 score:** The F1 score provides a balanced measure of detection performance by considering both recall and precision. This is particularly useful in imbalanced datasets, where accuracy alone may be misleading. A higher F1 score indicates a well-balanced IDS that effectively detects attacks while minimizing FPs, which make it a more reliable metric for evaluating overall detection performance.

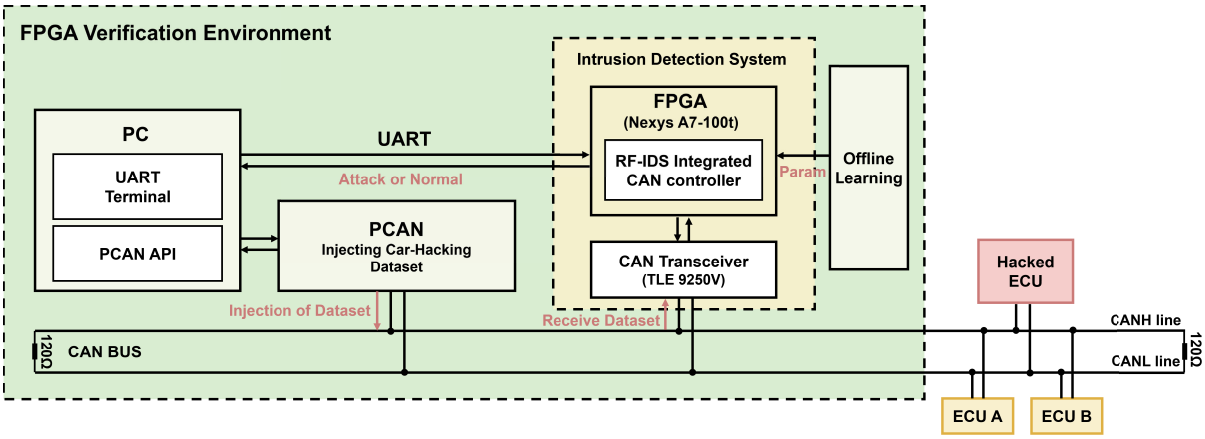


FIGURE 16. Block diagram of the RF-IDS verification environment.

TABLE 1. Car-hacking dataset.

Dataset	Normal messages	Attacking messages
DoS attack	3,078,250	587,521
Spoofing attack (Gear)	3,845,890	597,252
Spoofing attack (RPM)	3,966,805	654,897
Fuzzy attack	3,347,013	491,847

2) VERIFICATION ENVIRONMENT

Fig. 16 shows a block diagram illustrating the verification environment for an FPGA-based IDS system proposed in this study. This environment is designed to effectively replicate normal and attack traffic occurring within the in-vehicle CAN network and evaluate the detection performance of the proposed IDS.

The test dataset is injected into the CAN network connected to the FPGA via the PCAN API (Application Programming Interface), where the IDS processes the incoming traffic and determines the presence of an attack. Table 1 details the configuration of the Car-Hacking Dataset [25] used in this experiment, which contains both normal CAN messages and various types of attack messages. The dataset includes DoS, spoofing (gear and RPM), and fuzzy attacks, allowing the IDS to learn and evaluate its ability to detect multiple intrusion types.

The detection results of the IDS are processed in real time within the FPGA and transmitted to a personal computer via a universal asynchronous receiver transmitter interface. This setup enables a comprehensive evaluation of the detection performance and real-time response capabilities of the IDS.

Fig. 17 illustrates the verification of RF-IDS in a real FPGA environment using the Nexus A7-100T FPGA board. This figure demonstrates the process of handling the CAN traffic generated from an actual vehicle network and analyzing it using an IDS.

When an attack is detected, the RF-IDS performs "frame killing" by transmitting an error frame to abort the malicious CAN frame. Fig. 18 shows the oscilloscope measurement

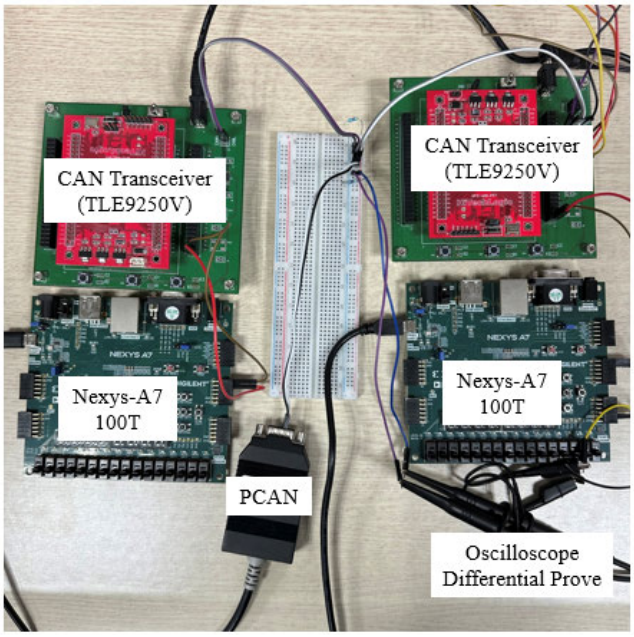


FIGURE 17. Actual FPGA verification environment.

results of the error flag generation process after the IDS detects an attack message. The left side of the figure shows a normal CAN frame, while the right side presents a sequence in which an error flag is generated after the IDS detects a message classified as an attack. On detecting an intrusion, the IDS immediately transmits an error frame, thereby ensuring that the malicious message is invalidated in the network and preventing other CAN nodes from receiving the attack frame. This confirms that the IDS can perform real-time attack detection and provide responses within a CAN.

3) EXPERIMENTAL RESULTS

Table 2 lists the detection performance of the IDS model for each attack type. The proposed model achieved the highest detection performance for DoS attacks while maintaining a

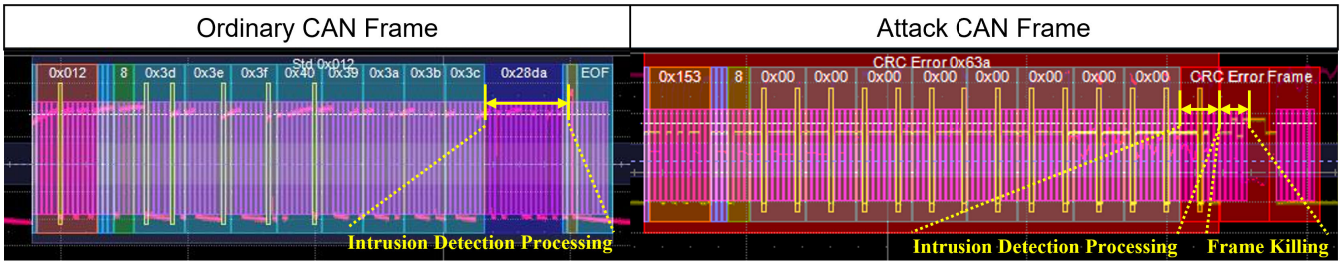


FIGURE 18. Oscilloscope waveform of normal and attack CAN frame.

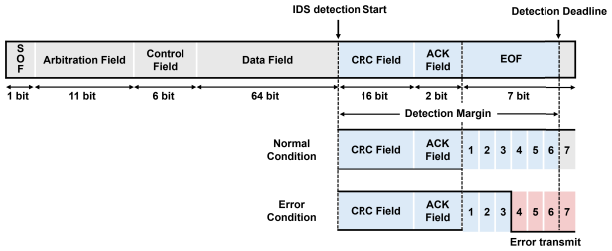


FIGURE 19. RF-IDS detection margin in the CAN frame.

TABLE 2. Performance comparison by attack types.

Attack type	Accuracy	Precision	Recall	F1 score
DoS	99.41	97.63	100.00	98.80
RPM (Spoofing)	99.25	97.66	97.59	97.63
Gear (Spoofing)	99.08	96.26	98.35	97.29
Fuzzy	98.70	94.94	99.12	96.98

stable detection performance for spoofing and fuzzy attacks. Although deep-learning-based IDS models offer high detection accuracy, they rely on large-scale computations that can lead to significantly higher hardware resource consumption and computational overhead. In contrast, the proposed model was designed with a more efficient hardware structure that optimizes both resource utilization and detection latency, thereby enabling real-time intrusion detection and blocking. Previous IDS models primarily focused on maximizing detection performance. however, many of them suffered from challenges in real-time response or edge device implementation because of their computational complexity. The IDS model proposed in this study is optimized for real-time operation in automotive environments, which helps minimize detection latency through hardware-level optimization while maintaining a high level of detection performance.

Fig. 19 illustrates the detection margin of the RF-IDS. In the CAN protocol, an error flag must be generated within six bits of the EOF when an attack is detected. If the detection time exceeds this limit, the IDS may detect an attack but fail to transmit the error flag in time, thereby making real-time blocking impossible.

In this study, the detection of random forest models was performed using an ID field. However, the RBF extended its detection capabilities by utilizing the data field to incorporate

TABLE 3. Detection latency comparison.

Model	Latency	Frames	Platform
MTH-IDS [13]	0.574 ms	per CAN frame	Raspberry Pi 3+
BNN-IDS [15]	0.259 ms	30 CAN frame	Zynq-7000
MA-QCNN [16]	0.43 ms	4 CAN frame	Zynq Ultrascale
DCNN [26]	5 ms	29 CAN frame	Tesla K80
QMLP-IDS [27]	0.24 ms	per CAN frame	Zynq Ultrascale
Proposed	**0.0018 ms**	**per CAN frame**	**Nexys-A7 100T**

TABLE 4. Hardware resource utilization comparison.

Model	Slice LUTs	Platform
QMLP-IDS [27]	56,733	Zynq-Ultrascale
MA-QCNN [16]	32,648	Zynq-Ultrascale
BNN-IDS [15]	23,256	Zedboard Zynq-7000
Proposed	**4,437**	**Nexys-A7 100T**

additional information. Accordingly, the RF-IDS initiates detection after the data field is received. Consequently, at the maximum CAN 2.0A transmission speed of 1 Mbps, the detection margin of RF-IDS is calculated as $1us/bit \times 24\ bit = 24\ us = 0.024\ ms$. In contrast, IDS models that rely solely on the ID field accelerate the detection start point, thereby allowing for an additional 64-bit free time window. This increased the total detection margin to 0.088 ms. however, as shown in Table 3 , existing IDS models exhibited long detection delays, which prevented them from meeting the required detection margin. This highlights their inability to achieve real-time responses despite their high detection accuracies.

However, the proposed RF-IDS achieved a detection delay of only 0.0018 *ms*, which helps ensure that it met the detection deadline and enabled real-time intrusion blocking in the CAN network.

Table 4 lists the look-up table (LUT) usages of the synthesized RF-IDS. The LUT is a fundamental logic component in FPGA design, which provides predefined outputs based on specific input-bit combinations. The proposed RF-IDS was designed to minimize FPGA resource utilization by adopting a low-computation architecture optimized for hardware implementation. Compared with other IDS models, the proposed system requires significantly fewer LUTs, effectively reducing hardware resource consumption. For example, QMLP-IDS requires 56,733 LUTs, while the

TABLE 5. ASIC implementation of proposed IDS.

Metric	Proposed IDS
Using system clock	50 MHz
Random forest attack detection (IDS)	32,051 gate
CAN controller and frame killing (IPS)	40,625 gate
Random forest coefficient SRAM memory	167,030 gate
Rule-based filter and final judgement	5,516 gate

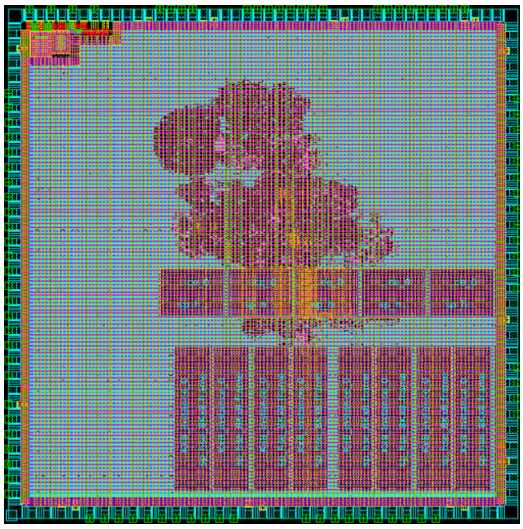


FIGURE 20. Chip layout of RF-IDS.

proposed model achieves similar detection capabilities using only 4,437 LUTs, which represent an approximately 12-fold reduction in resource usage.

4) ASIC IMPLEMENTATION

In this study, the ASIC design was implemented using the TSMC 180 nm process to verify the feasibility of the actual chip implementation for the proposed IDS. As a result of synthesizing the IDS circuit, the total gate count, which includes the CAN controller, was measured at 245,222 gates, and five 4 KB SRAMs were used for data storage and processing within the IDS. Table 5 summarizes the ASIC design results, which confirm that the proposed IDS operates on a 50 MHz system clock while efficiently utilizing hardware resources. This design ensures that real-time detection can be performed without excessive hardware consumption.

Fig. 20 and Fig. 21 show the chip layout and test environment of the proposed model, respectively. The IDS was successfully implemented as an ASIC through the MPW process because of its low hardware resource usage, thereby demonstrating that it enables real-time detection while maintaining high performance in a resource-constrained environment.

VI. LIMITATION AND DISCUSSION

The proposed RF-IDS effectively performs detection and prevention in CAN bus environments. However, the system also has limitations that constrain its adaptability and scalability. The proposed IDS has limitations in responding to previously unknown or new attack types. As a supervised learning algorithm, the random forest model fundamentally depends on pre-labeled datasets and lacks the capability for unsupervised anomaly detection. Furthermore, the primary objective of this study is to design a hardware-implemented IDS capable of real-time operation on the CAN bus. Accordingly, the proposed architecture is designed to detect trained attack patterns that can be processed in real time using the random forest model. As a result, the system may fail to identify new intrusion patterns that were not present during the training phase. To overcome these limitations, future research is needed to develop a collaborative IDS architecture that operates across cloud and on-device environments. In this extended system, the cloud-side IDS could handle new attack types and unsupervised anomaly detection in a non-real-time method. In contrast, the on-device IDS handles real-time detection of critical, well-known attacks. Such a hybrid approach would combine the flexibility of software-based detection with the responsiveness of hardware implementation, resulting in a more robust and adaptive security framework for in-vehicle networks.

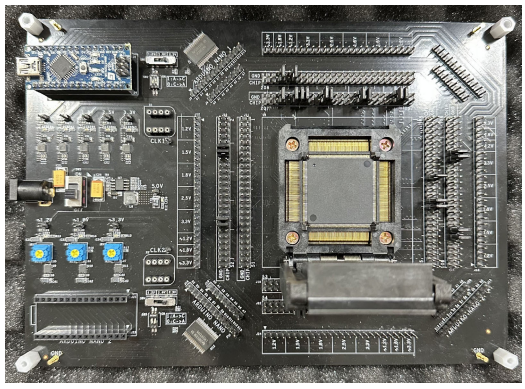


FIGURE 21. Chip test environment.

VII. CONCLUSION

A lightweight real-time CAN bus IDS based on random forest was proposed in this study. The random forest model was trained using both normal and attack data and designed to perform detection through simple comparison operations, which eliminated the need for complex matrix computations. In addition, the system achieved an average detection delay of 1.86 μs by integrating the IDS model with the CAN controller, thereby demonstrating its ability to block attack messages in real time.

Analyzing the detection margin revealed that the proposed IDS was capable of real-time detection even at the maximum CAN transmission speed (1 Mbps), while the existing IDS models failed to meet the required timing constraints. Through hardware optimization, the IDS was implemented with significantly fewer resources than the previous designs,

utilizing only 4,437 LUTs, which made it suitable for deployment in resource-constrained environments. Finally, the IDS was implemented as an ASIC using the TSMC 180 nm process with a total gate count of 245,222 gates, including the CAN controller. This study confirmed that a random forest-based IDS was a viable solution for automotive security systems because it delivered a high detection performance and real-time response, even with limited hardware resources.

In future work, we plan to further optimize the detection performance by refining the model design to improve hardware efficiency. Furthermore, we intend to explore a hybrid IDS architecture that integrates both cloud and on-device components.

ACKNOWLEDGMENT

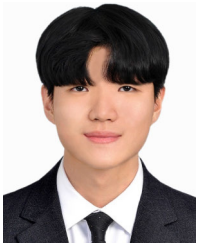
The CAD tool was supported by the IC Design Education Center (IDEC). *(Junseok Lee and Sangmin Park contributed equally to this work.)*

REFERENCES

[1] *Future Advances in Body Electronics*. Accessed: Mar. 22, 2013. [Online]. Available: https://www.nxp.com/docs/en/white-paper/BODYDELECTRWP.pdf
[2] H. Jeong, Y. Shen, J. Jeong, and T. T. Oh, "A comprehensive survey on vehicular networking for safe and efficient driving in smart transportation: A focus on systems, protocols, and applications," *Veh. Commun.*, vol. 31, Oct. 2021, Art. no. 100349.
[3] C. Miller, and C. Valasek, "Remote exploitation of an unaltered passenger vehicle," in *Proc. Black Hat USA*, Aug. 2015, pp. 1–91.
[4] X. Mo, P. Chen, J. Wang, and C. Wang, "Anomaly detection of vehicle CAN network based on message content," in *Proc. Int. Conf. Secur. Privacy New Comput. Environ.*, Jun. 2019, pp. 96–104.
[5] S. Checkoway, D. McCoy, B. Kantor, D. Anderson, H. Shacham, and S. Savage, K. Koscher, A. Czeskis, F. Roesner, and T. Kohno, "Comprehensive experimental analyses of automotive attack surfaces," in *Proc. USENIX Secur. Symp.*, 2011, pp. 77–92.
[6] Z. Yu, Y. Liu, G. Xie, R. Li, S. Liu, and L. T. Yang, "TCE-IDS: Time interval conditional entropy- based intrusion detection system for automotive controller area networks," *IEEE Trans. Ind. Informat.*, vol. 19, no. 2, pp. 1185–1195, Feb. 2023.
[7] S. Li, Y. Cao, H. J. Hadi, F. B. Hussain, L. Chen, and F. Hao, "ECF-IDS: An enhanced cuckoo filter-based intrusion detection system for in-vehicle network," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 4, pp. 3456–3467, Aug. 2024, doi: 10.1109/TNSM.2024.3394842.
[8] J. A. Khan, D.-W. Lim, and Y.-S. Kim, "A deep learning-based IDS for automotive theft detection for in-vehicle CAN bus," *IEEE Access*, vol. 11, pp. 112814–112829, 2023.
[9] H. Jo and D.-H. Kim, "Intrusion detection using transformer in controller area network," *IEEE Access*, vol. 12, pp. 121932–121946, 2024.
[10] E. S. Seo, J. E. Kim, W. Lee, and J. Seok, "Adversarial attack of ML-based intrusion detection system on in-vehicle system using GAN," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 3, pp. 540–555, May 2021.
[11] M. H. Bhavsar, Y. B. Bekele, K. Roy, J. C. Kelly, and D. Limbrick, "FL-IDS: Federated learning-based intrusion detection system using edge devices for transportation IoT," *IEEE Access*, vol. 12, pp. 52215–52226, 2024, doi: 10.1109/ACCESS.2024.3386631.
[12] L. Yang, A. Moubayed, and A. Shami, "MTH-IDS: A multitiered hybrid intrusion detection system for Internet of Vehicles," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 616–632, Jan. 2022.
[13] H. M. Song, H. R. Kim, and H. K. Kim, "Intrusion detection system based on the analysis of time intervals of CAN messages for in-vehicle network," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2016, pp. 63–68.
[14] A. Rangsikunpum, S. Amiri, and L. Ost, "An FPGA-based intrusion detection system using binarised neural network for CAN bus systems," in *Proc. IEEE Int. Conf. Ind. Technol. (ICIT)*, Mar. 2024, pp. 1–6.
[15] S. Khandelwal and S. Shreejith, "A lightweight multi-attack CAN intrusion detection system on hybrid FPGAs," in *Proc. 32nd Int. Conf. Field-Programmable Log. Appl. (FPL)*, Aug. 2022, pp. 425–429.
[16] R. B. GmbH, "CAN specification," Version 2.0, 1991. [Online]. Available: http://esd.cs.ucr.edu/webres/can20.pdf
[17] *Road Vehicles–Controller Area Network (CAN)—Part 2: High-Speed Medium Access Unit*, ISO Standard 11898-2, 2016.
[18] I. Foster, A. Prudhomme, K. Koscher, and S. Savage, "Fast and vulnerable: A story of telematic failures," in *Proc. 9th USENIX Workshop Offensive Technol. (WOOT)*, Aug. 2015, pp. 1–11.
[19] S. B. Park, H. J. Jo, and D. H. Lee, "Flooding attack mitigator for in-vehicle CAN using fault confinement in CAN protocol," *Proc. Comput. Secur.*, vol. 126, Mar. 2023, Art. no. 103091.
[20] K. Iehira, H. Inoue, and K. Ishida, "Spoofing attack using bus-off attacks against a specific ECU of the CAN bus," in *Proc. 15th IEEE Annu. Consum. Commun. Netw. Conf. (CCNC)*, Jan. 2018, pp. 1–4.
[21] K. Koscher, A. Czeskis, F. Roesner, S. Patel, T. Kohno, S. Checkoway, D. McCoy, B. Kantor, D. Anderson, H. Shacham, and S. Savage, "Experimental security analysis of a modern automobile," in *Proc. IEEE Symp. Secur. Privacy*, May 2010, pp. 447–462.
[22] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, Oct. 2001.
[23] K. Mouratidis and D. Papadias, "Continuous nearest neighbor queries over sliding windows," *IEEE Trans. Knowl. Data Eng.*, vol. 19, no. 6, pp. 789–803, Jun. 2007.
[24] H. Im, D. Lee, and S. Lee, "A novel architecture for an intrusion detection system utilizing cross-check filters for in-vehicle networks," *Sensors*, vol. 24, no. 9, p. 2807, Apr. 2024.
[25] *HCRL—Car-Hacking Dataset*. Accessed: Dec. 13, 2024. [Online]. Available: https://ocslab.hksecurity.net/Datasets/car-hacking-dataset
[26] H. M. Song, J. Woo, and H. K. Kim, "In-vehicle network intrusion detection using deep convolutional neural network," *Veh. Commun.*, vol. 21, Jan. 2020, Art. no. 100198, doi: 10.1016/j.vehcom.2019.100198.
[27] S. Khandelwal and S. Shreejith, "A lightweight FPGA-based IDS-ECU architecture for automotive CAN," in *Proc. Int. Conf. Field-Program. Technol. (ICFPT)*, Hong Kong, Dec. 2022, pp. 1–9.



JUNSEOK LEE received the B.S. degree from the School of Electronic Engineering, Soongsil University, Seoul, South Korea, in 2023. He is currently pursuing the M.S. degree with the Department of Intelligent Semiconductors. His research interests include vehicle security, artificial intelligence, hardware accelerators, and automotive SoC.



SANGMIN PARK received the B.S. degree from the School of Electronic Engineering, Soongsil University, Seoul, South Korea, in 2024. He is currently pursuing the M.S. degree with the Department of Intelligent Semiconductors. His research interests include vehicle security, artificial intelligence, hardware accelerators, and automotive SoC.



SUA SHIN received the B.S. degree from the School of Electronic Engineering, Soongsil University, Seoul, South Korea, in 2024. She is currently pursuing the M.S. degree with the Department of Intelligent Semiconductors. Her research interests include vehicle security, artificial intelligence, hardware accelerators, and automotive SoC.



HYUNGCHUL IM (Graduate Student Member, IEEE) received the B.S. degree from the School of Mechanical Engineering, Soongsil University, Seoul, South Korea, in 2021. He is currently pursuing the Ph.D. degree with the Department of Intelligent Semiconductors. His research interests include vehicle security, artificial intelligence, and automotive SoC.



JOOSOCK LEE received the B.S. degree in electronic engineering from Sogang University in 1983, the M.S. and Ph.D. degrees in electronic engineering from Korea University, South Korea, in 1985 and 1999, respectively. From 1984 to 1995, he was a Research Engineer at LG Electronics R&D Center. From 2004 to 2006, he led the development of SoCs for mobile phones as the CTO at MtekVision Ltd. Since 2022, he has been with Soongsil University, where he is currently a Professor with the Department of Intelligent Semiconductors. His research interests include AI SoC, automotive SoC, and power SoC.



SEONGSOO LEE (Member, IEEE) received the B.S., M.S., and Ph.D. degrees in electronic engineering from Seoul National University, South Korea. From 1998 to 2000, he was a Research Associate at the University of Tokyo, Japan. From 2000 to 2002, he was a Research Professor at Ewha Womans University, South Korea. Since 2002, he has been with Soongsil University, where he is currently a Professor with the Department of Intelligent Semiconductors. Since 2023, he has been as the President of the Institute of Korean Electrical and Electronics Engineers, South Korea. His research interests include AI SoC, processor SoC, security SoC, automotive SoC, and power SoC.